

Interactive R tutorials in a browser environment: tools, templates and lessons for teaching and assessing data analysis

Léo Belzile¹ , David A. Campbell² , Kyran Cupido³ , Nishan Mudalige⁴ , Aurélien Nicosia⁵, Emily Somerset⁴, Haochen Song⁴ , David Riegert⁶, Russell Steele⁷ , Irene Vrbik⁸, Karen Huynh Wong⁴

¹ HEC Montréal, leo.belzile@hec.ca

² Carleton University, davecampbell@math.carleton.ca

³ St. Francis Xavier University, kcupido@stfx.ca

⁴ University of Toronto, nishan.mudalige@utoronto.ca; emily.somerset@mail.utoronto.ca; fred.song@mail.utoronto.ca; karen.huynhwong@utoronto.ca

⁵ Université Laval, aurelien.nicosia@mat.ulaval.ca

⁶ Trent University, davidriegert@trentu.ca

⁷ McGill University, russell.steele@mcgill.ca

⁸ University of British Columbia, irene.vrbik@ubc.ca

Abstract

Interactive coding exercises allow students to practise data analysis and receive immediate feedback, but their deployment has traditionally required either local software installation or a server running R. WebAssembly builds of R (webR) remove both requirements: R code runs in the student’s web browser, and course materials can be served from any static web host. We review software for teaching, assessing and grading data analysis skills in R, with emphasis on webR, the Quarto extensions that embed it in documents and slides, `learnr` and `gradethis`. We describe an open course skeleton, developed at the 2025 Canadian Conference on Teaching Statistics (CanCOTS), that combines lecture slides, simulation activities, tutorials and auto-graded exercises running entirely in the browser. Using this template, we illustrate how exercises with hints, solutions and automated checks are written, discuss the uses and limitations of the approach, and offer recommendations for designing checks that provide informative feedback.

Keywords: statistics education, data science education, WebAssembly, Quarto, formative assessment, reproducible teaching

1 Introduction

Data science majors popularity is at an all time high ([Pierson 2023](#)), leading to strain on teaching resources. The increased enrollment, even in upper level undergraduate courses, increases the time spent on class management by instructors and limits their ability and that of teaching assistants to provide timely feedback on evaluations. The advent of generative artificial intelligence (genAI) also raises concerns about plagiarism. Due to these, instructors more and more default back to invigilated multiple choice exams. This is unfortunate because the questions that can be asked in such evaluations are not necessarily authentic.

Data science is interdisciplinary by nature: standard tasks performed by analysts include exploratory data analysis and data wrangling, visualisation, model specification, estimation and validation, along with reporting. Designing assignments should therefore try to emulate those tasks while fostering statistical thinking ([Woodard et al. 2020](#)). For formative and summative assessment, it is often beneficial to have clear and consistent grading rubrics, which facilitate learning experience ([Poulos & Mahony 2008](#)). Automating feedback by parsing code, and using natural language processing (NLP) for comments can lead to efficiency gains ([Galassi & Vittorini 2021](#)), while software solutions which facilitate sharing of grading rubrics, such as e.g. and Otter-Grader ([Pyles 2025](#)), ensure some consistency across instructors. Version control is now often covered as part of the content in an introductory data science courses (e.g., [Timbers et al. 2022](#)), and it is also becoming more frequent for class management ([Rundel & Çetinkaya-Rundel 2026](#), [Ricci et al. 2024](#)). Solutions such as WebWork ([Jahodová Berková & Němec 2020](#), [Zimmerman 2020](#)) or R-exams ([Zeileis et al. 2014](#)) give instructors tools to effectively manage large-scale assessments using randomized questions databases. The automation of many of these grading and feedback tasks remains

difficult because, while there are wrong answers, there is no unique correct one. Thus, setting up rubrics requires much upfront work to determine most likely errors.

Our purpose is to review software tools for teaching, assessing and grading data analysis skills in the classroom using the **R** programming language ([R Core Team 2025](#)), a widely-used interpreted language which is popular among statisticians. There exists a wealth of contributed packages and software bundles to encourage semi-automation of tasks, generation of randomized evaluations, and tutorials. One key point is keeping the technical barriers low also for instructors, who can easily adapt their existing examples to a more interactive format. We consider several examples, notably:

- In-class activities with limited entry-level barriers to programming via [webR](#) coupled with [Quarto Live](#);
- Interactive tutorials via the `learnR` ([Aden-Buie, Schloerke, Allaire & Rossell Hayes 2025](#), [Stoudt et al. 2022](#)) package, including description of randomization within, with graduated hints and support for multiple solutions [gradethis](#) and recovery of student responses using `submitr` or `learnhash`.
- Semiautomatic tools for extracting student solutions for coding tasks and autograding, focusing on invariance of outputs.

The purpose of this paper is to review these three aspects critically with different points in mind and to

- Introduce, review and highlight these tools (e.g., limitations in `learnr` for randomization due to pre-rendering)
- Suggest strategies for building unit tests and enabling informative feedback
- Streamline the use of these tools by providing templates for users

- Report on instructors experience implementing these tools in their classroom during the 2025–2026 academic year ???.

The remainder of the paper is organized as follows: Section 2 compares the main tools; Section 3 describes an open course skeleton built with webR (CanCOTS 2025 Interactive Tutorials Working Group 2025), its requirements and an example of auto-graded exercises, and Section 4 summarizes its strengths and limitations; Section 5 and Section 6 discuss `learnr` tutorials and the autograding of R Markdown and Quarto documents, Section 7 gives recommendations for designing tutorials and checks, and Section 8 reviews evidence from the classroom; Section 9 concludes with a discussion, and Section 10 briefly describes a possible extension using large language models [Should we include this?]

2 Tools for interactive R tutorials

2.1 Description of tools

Table 1: Description of the main tools reviewed in this paper.

Tool	Description	Hosting	Related packages
webR	A version of R compiled for the browser and Node.js using WebAssembly. It allows R code to run in the user’s browser without an R server (Stagg et al. 2023b).	Can work with static web hosting because computation happens client-side, although package support depends on WebAssembly-compatible R packages (Stagg et al. 2023a,b).	Useful for lightweight, browser-based R interactivity. It is conceptually different from standard learnr , which usually depends on Shiny.
learnr	An R package for turning R Markdown documents into interactive tutorials with narrative, code exercises, quizzes, videos, and Shiny components (Aden-Buie, Schloerke, Allaire & Rossell Hayes 2025).	Standard learnr tutorials use runtime::shiny_prerendered , so they usually need to run locally or on a Shiny-capable hosting platform rather than as ordinary static pages (Aden-Buie, Schloerke, Allaire & Rossell Hayes 2025).	Provides the tutorial framework that can incorporate gradethis feedback and, in some workflows, learnrhash tracking.

Tool	Description	Hosting	Related packages
gradethis	An R package that provides automated feedback for student exercises in learnr tutorials. It can compare student code to a model solution or use custom grading logic (Aden-Buie, Chen, Grolemond, Rossell Hayes & Schloerke 2025).	Its deployment constraints generally follow learnr , since it is designed to run inside learnr tutorials (Aden-Buie, Chen, Grolemond, Rossell Hayes & Schloerke 2025).	Extends learnr by adding automated feedback to code exercises.
Quarto	An open-source scientific and technical publishing system for documents, websites, blogs, books, presentations, dashboards, and other outputs (Allaire et al. 2026).	Can produce static HTML outputs that are suitable for static hosting; dynamic or live interactive content may require an appropriate runtime or hosting environment (Allaire et al. 2026).	Useful for publishing course materials, while learnr is more specialized for interactive tutorials.

Tool	Description	Hosting	Related packages
Other tools	Tools discussed elsewhere in this paper: the Quarto extensions <code>quarto-webr</code> (Balamuta 2023) and <code>quarto-live</code> (Stagg 2024); <code>learnrhash</code> and <code>submitr</code> for collecting <code>learnr</code> responses; R-exams and WeBWorK for randomized assessments; Otter-Grader and <code>ghclass</code> for autograding and class management.	The Quarto extensions support static hosting. <code>learnrhash</code> and <code>submitr</code> require Shiny, as for <code>learnr</code> . WeBWorK requires its own server; the remaining tools run on the instructor's machine.	The Quarto extensions build on <code>webR</code> ; <code>learnrhash</code> and <code>submitr</code> extend <code>learnr</code> ; Otter-Grader supports R through <code>ottr</code> .

2.2 Use cases

Table 2: Use cases, strengths and limitations of each tool.

Tool	Use case summary	Strengths	Limitations / comparison
webR	Use when students need to run R code directly in the browser without installing R or connecting to an R server (Stagg et al. 2023b).	Strong for static or mostly-static websites because computation happens client-side. Useful for lightweight browser-based coding activities, demos, and interactive examples.	Less mature than standard server-side R workflows. The API is under active development, some packages may not work, and mobile browsers may have memory limits (Stagg et al. 2023b). Compared with learnr , it is better for static hosting but less specialized for structured tutorials and feedback.

Tool	Use case summary	Strengths	Limitations / comparison
learnr	Use when you want a guided interactive tutorial with narrative, editable code exercises, quizzes, videos, and Shiny components (Aden-Buie, Schloerke, Allaire & Rossell Hayes 2025).	Strong for self-paced active learning. Gives students a structured environment where they read, run code, answer questions, and receive feedback. Works well with <code>gradethis</code> .	Standard learnr tutorials use <code>runtime::shiny_prerendered</code> , so they usually need a Shiny-capable environment rather than ordinary static hosting (Aden-Buie, Schloerke, Allaire & Rossell Hayes 2025). Compared with Quarto, it is more specialized for tutorials but less general for course websites, slides, books, or reports.

Tool	Use case summary	Strengths	Limitations / comparison
gradethis	Use when you want automated feedback on student code inside a learnr tutorial (Aden-Buie, Chen, Grolemond, Rossell Hayes & Schloerke 2025).	Strong for formative assessment. Can compare student code to a model solution or use custom grading logic, which helps give targeted feedback for common mistakes (Aden-Buie, Chen, Grolemond, Rossell Hayes & Schloerke 2025).	It is not a general publishing tool or a standalone assessment system. Its use is closely tied to learnr , so its hosting constraints usually follow learnr . Compared with learnr , it is not the tutorial framework itself; it is the feedback layer.

Tool	Use case summary	Strengths	Limitations / comparison
Quarto	Use when you want to create course notes, reports, slides, websites, blogs, books, dashboards, or other reproducible teaching materials (Allaire et al. 2026).	Strong for publishing. Supports multiple languages, including R, Python, Julia, and Observable, and can produce many output formats such as HTML, PDF, Word, slides, websites, and books (Allaire et al. 2026).	It is not primarily an interactive tutorial or grading system. Static outputs are easy to host, but live computation or rich interactivity may require additional tools or hosting. Compared with <code>learnr</code> , Quarto is broader and better for publishing; <code>learnr</code> is better for guided coding tutorials.

3 An Introduction to running R in the browser using webR

3.1 A Course Template for Beginners

At the 2025 Canadian Conference on Teaching Statistics (CanCOTS, Montréal, 10–12 June 2025), the interactive tutorials working group assembled a template for a complete course in which every interactive component runs R in the student’s browser ([CanCOTS 2025 Interactive Tutorials Working Group 2025](#)). The template is a Quarto website ([Allaire et al. 2026](#)) consisting of a landing page, a syllabus, lecture slides, simulation activities, tutorials and a graded tutorial, and is published with GitHub Pages. Instructors may fork the repository, replace the content with their own and publish their copy in the same manner.

The materials in the template were contributed by several of the authors: the course skeleton and lecture slides by I. Vrbik, the activities and simulations by K. Cupido, and the tutorials by N. Mudalige. The graded tutorial shown in Figures 6 to 9 is taken from the STA258 tutorial website at the University of Toronto Mississauga (https://nishanmudalige.github.io/STA258_Tutorials/), developed by N. Mudalige. The remaining screenshots were taken from the live template site in a desktop browser.

3.2 How webR works

webR is a version of the R interpreter compiled to WebAssembly, a binary format executed by modern web browsers ([Stagg et al. 2023b](#)). When a page is opened, the browser downloads the webR runtime, starts an R session and installs the packages declared by the

page. Packages must be compiled to WebAssembly; a public repository provides builds for a large share of CRAN, and other packages can be compiled with `rwasm` (Stagg et al. 2023a). Since all computation takes place on the student’s device, the server only delivers static files, and any static host is sufficient.

webR is embedded in documents through Quarto extensions that convert code chunks into editable cells with a “Run Code” button. The template uses two such extensions:

- `quarto-webr` (Balamuta 2023) provides `{webr-r}` cells for HTML documents and revealjs slides, and is used for the slides, activities and ungraded tutorials. Packages are declared in the YAML header of each page (such as in Listing 1), and chunk options determine whether a cell runs automatically (`autorun`) or prepares the session (`context: setup`).
- `quarto-live` (Stagg 2024) provides the `live-html` format with `{webr}` cells and supports exercises with hints, solutions and checks written with `gradethis` (Aden-Buie, Chen, Grolemond, Rossell Hayes & Schloerke 2025). It is used for the graded tutorial.

Listing 1 YAML header of a page in the template that uses the `quarto-webr` extension.

The `webr` filter enables interactive R cells, and the listed packages are installed in the browser when the page is opened.

```
format:
  html:
    toc: true
filters:
  - webr
webr:
  packages: ["tibble", "ggplot2"]
```

Both extensions are included in the repository, so no further installation is needed to render the site.

3.3 Requirements

Instructors require R and Quarto (and optionally RStudio) to edit and render the materials, and a GitHub account or another static host to publish them. No Shiny server is involved. Students require only a recent desktop web browser and an internet connection; no software or account is needed. On the first visit, the browser downloads the R runtime and the packages. In our tests, the coin-flipping activity downloaded approximately 55 MB and was ready after approximately 20 seconds, whereas the graded tutorial, which also loads `gradethis`, required approximately one minute. These times depend on the network and the device. Loading progress is displayed on each page (Figure 1), and mobile browsers may have insufficient memory for larger pages (Stagg et al. 2023b).

An Activity in R on Coin Flipping

Run R code to simulate the results of flipping coins many times

AUTHOR
Kyran Cupido

AFFILIATION
St. Francis Xavier University

WEBR STATUS
🔄 Installing package 2 out of 2: ggplot2

Coin Flip Example

For a random variable X that follows the Bernoulli distribution with probability p , each realization of the random variable has two possible outcomes (e.g., Success or Failure, 1 or 0, Heads or Tails, etc.). More specifically, the Bernoulli distribution is a discrete probability distribution such that $P(X = 1) = p$ and $P(X = 0) = 1 - p$.

As a simple example, we can design an experiment where we flip a coin and record the outcome using the following code:

```

1 coin <- c("Heads", "Tails")
2 sample(coin, 1)

```

(a) While loading

An Activity in R on Coin Flipping

Run R code to simulate the results of flipping coins many times

AUTHOR
Kyran Cupido

AFFILIATION
St. Francis Xavier University

WEBR STATUS
● Ready!

Coin Flip Example

For a random variable X that follows the Bernoulli distribution with probability p , each realization of the random variable has two possible outcomes (e.g., Success or Failure, 1 or 0, Heads or Tails, etc.). More specifically, the Bernoulli distribution is a discrete probability distribution such that $P(X = 1) = p$ and $P(X = 0) = 1 - p$.

As a simple example, we can design an experiment where we flip a coin and record the outcome using the following code:

```

1 coin <- c("Heads", "Tails")
2 sample(coin, 1)

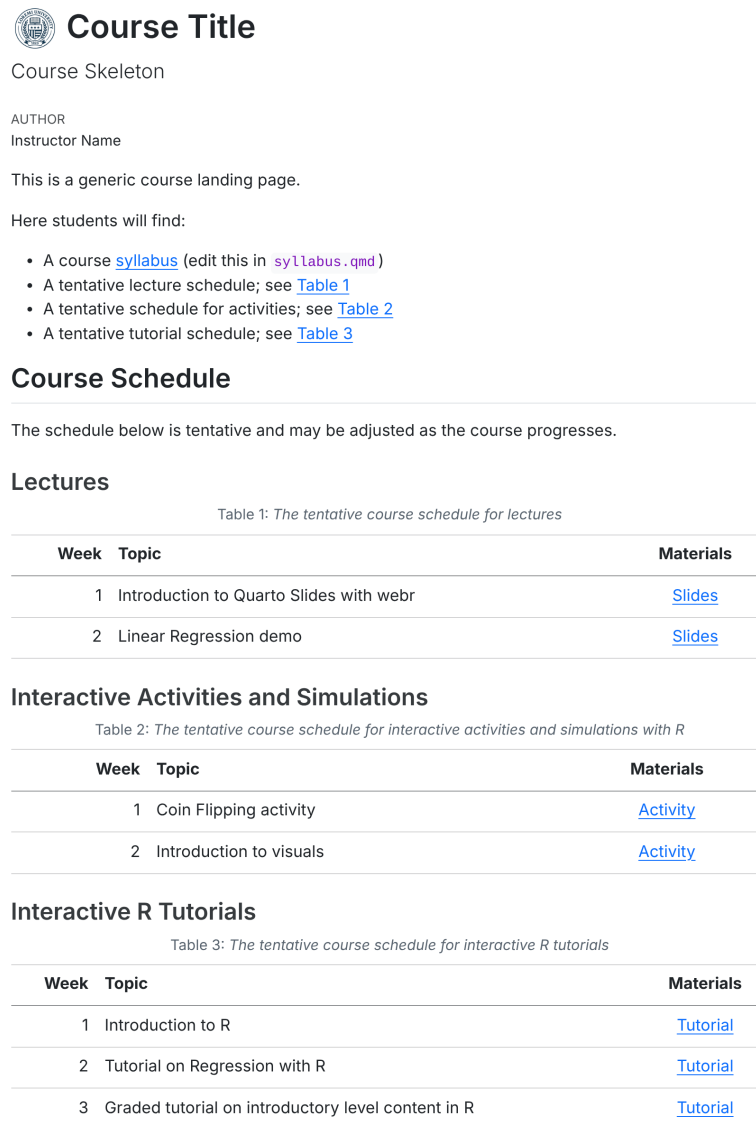
```

(b) Ready

Figure 1: The coin-flipping activity (a) while webR installs the declared packages and (b) once the R session is ready.

3.4 Structure of the template

The landing page (Figure 2) links to the syllabus and to schedules of lecture slides, activities and tutorials. Each item is a separate Quarto document, so weeks are added or removed by editing a table.



 **Course Title**

Course Skeleton

AUTHOR
Instructor Name

This is a generic course landing page.

Here students will find:

- A course [syllabus](#) (edit this in `syllabus.qmd`)
- A tentative lecture schedule; see [Table 1](#)
- A tentative schedule for activities; see [Table 2](#)
- A tentative tutorial schedule; see [Table 3](#)

Course Schedule

The schedule below is tentative and may be adjusted as the course progresses.

Lectures

Table 1: The tentative course schedule for lectures

Week	Topic	Materials
1	Introduction to Quarto Slides with webr	Slides
2	Linear Regression demo	Slides

Interactive Activities and Simulations

Table 2: The tentative course schedule for interactive activities and simulations with R

Week	Topic	Materials
1	Coin Flipping activity	Activity
2	Introduction to visuals	Activity

Interactive R Tutorials

Table 3: The tentative course schedule for interactive R tutorials

Week	Topic	Materials
1	Introduction to R	Tutorial
2	Tutorial on Regression with R	Tutorial
3	Graded tutorial on introductory level content in R	Tutorial

Figure 2: Landing page of the course skeleton.

Lecture slides. The slides are revealjs presentations with embedded `{webr-r}` cells, which allow code to be run and modified during the lecture without leaving the slides (Figure 3).

The same deck can be printed to PDF.

Recall: Scatterplots

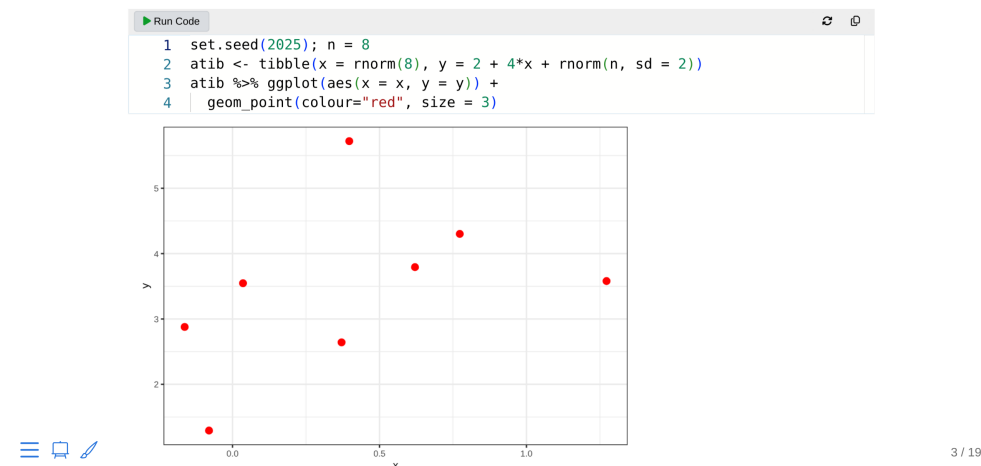


Figure 3: A slide from the regression lecture after running the code. Tall plots may be cut off at the bottom of the slide, as is the case for the x -axis label here.

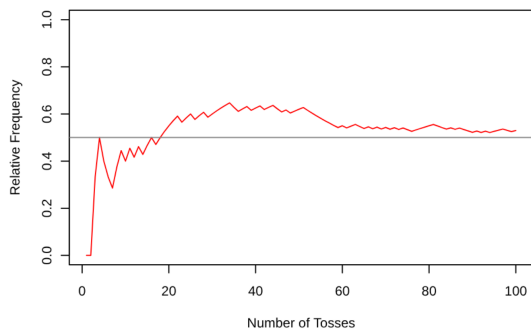
Activities and simulations. Activities combine short explanations with code cells that students modify and re-run. The coin-flipping activity builds up to a simulation of the running proportion of heads (Figure 4a), and the Pixar activity uses the `pixarfilms` package for descriptive statistics and basic graphs (Figure 4b).

```

1 num_flips <- 100
2 flips <- sample(coin, size = num_flips,
3   replace = TRUE)
4 heads_freq <- cumsum(flips == 'Heads') /
5   1:num_flips
6 plot(heads_freq,
7   type = 'l',
8   col="red",
9   xlab = "Number of Tosses",
10  ylab="Relative Frequency",
11  ylim = c(0, 1))
12 abline(h = 0.5, col = 'gray50')

```

Download Image



(a) Coin flipping

```

1 mean(run_time)
[1] 100.6522

```

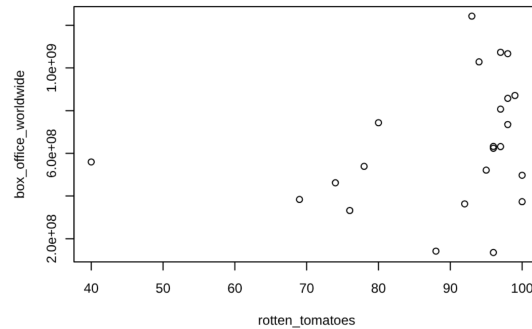
Basic Plots

There is a generic `plot` function in R which is useful for creating quick visualizations of a data set. If you pass one numeric variable into this function, a quick plot will be made by creating a point for each observation in the order they occur in the data frame (index). Passing two numeric variables into this function will create a scatterplot, with the first variable corresponding to the x-axis and the second corresponding to the y-axis.

```

1 plot(rotten_tomatoes, box_office_worldwide)

```



(b) Pixar films

Figure 4: Code cells from the two activities after running the code.

Tutorials. The ungraded tutorials follow a worked-example format. In the regression tutorial, students explore the Palmer penguins data ([Horst et al. 2020](#)) and fit a simple linear regression with `lm()` (Figure 5). The R session persists across the cells of a page, as in an ordinary R script.

```

▶ Run Code
1 Adelie_lm = lm(flipper_length_mm ~
2   body_mass_g, data = Adelie)
3 summary(Adelie_lm)

Call:
lm(formula = flipper_length_mm ~ body_mass_g, data = Ad

Residuals:
    Min       1Q   Median       3Q      Max
-14.426  -3.669   0.239   3.422  17.955

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept) 1.656e+02   3.918e+00   42.27  < 2e-16 ***
body_mass_g  6.610e-03   1.049e-03    6.30  3.4e-09 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.

Residual standard error: 5.794 on 144 degrees of freedom
Multiple R-squared:  0.2161,    Adjusted R-squared:  0.2
F-statistic: 39.69 on 1 and 144 DF,  p-value: 3.402e-09

```

Figure 5: A cell from the regression tutorial. Long output lines are truncated at the edge of the cell.

3.5 Example of use: graded exercises

Each exercise in the graded tutorial consists of up to five chunks that share an **exercise** label: an optional setup chunk, the starting code, a hint, a model solution and a check. Listing 2 shows the source of the first exercise, which asks students to replace two blanks so that an expression evaluates to 10. The blanks are written with a full-width underscore character (`_`), which the setup chunk defines as a missing value.

Listing 2 Quarto source of the first exercise of the graded tutorial, consisting of a setup chunk, the starting code, a hint, a model solution and a check written with gradethis.

```
```{webr}
#| setup: true
#| exercise: ex_sum
_ <- NA_real_
```

```{webr}
#| exercise: ex_sum
1 + 2 + _ + _
```

::: {.hint exercise="ex_sum"}
Replace the two _ with 2 numbers that add up to 7.
:::

::: {.solution exercise="ex_sum"}
```{webr}
#| exercise: ex_sum
#| solution: true
1 + 2 + 3 + 4
```
:::

```{webr}
#| exercise: ex_sum
#| check: true
gradethis::grade_this({
 if (grepl("_", .user_code, fixed = TRUE)) {
 fail("Replace _ with a number.")
 }
 if (is.numeric(.result) && length(.result) == 1 &&
 isTRUE(all.equal(.result, 10))) {
 pass("✓ Correct!")
 } else {
 fail("Not quite - try to make the expression evaluate to 10.")
 }
})
```
```

Within `grade_this()`, the check has access to the student's code as text (`.user_code`), the

value of the last expression (`.result`) and the evaluation environment (`.envir_result`), and returns feedback through `pass()` or `fail()`. Checks are therefore ordinary R code. Running the code displays its output followed by the feedback from the check (Figure 6). Once the hint has been revealed, the hint button is replaced by a “Show Solution” button, so that assistance is provided in stages.

Q1 — Make the sum equal 10

Replace `_` with a number so the expression evaluates to **10**.

Exercise

[↺ Start Over](#) [⚠ Show Solution](#)

[▶ Run Code](#)

```
1 1 + 2 + 3 + 5
```

[1] 11

Not quite — try to make the expression evaluate to 10.

Replace the two `_` with 2 numbers that add up to 7.

(a) Incorrect answer, after revealing the hint

Q1 — Make the sum equal 10

Replace `_` with a number so the expression evaluates to **10**.

Exercise

[↺ Start Over](#) [⚠ Show Solution](#)

[▶ Run Code](#)

```
1 1 + 2 + 3 + 4
```

[1] 10

✓ Correct!

Replace the two `_` with 2 numbers that add up to 7.

(b) Correct answer

Figure 6: The first exercise of the graded tutorial, with (a) an incorrect answer and the hint and (b) the corrected answer.

Later exercises inspect the code itself. For a question requiring a histogram of one column of `ToothGrowth`, the check parses the submission with `parse()` and verifies that it is a

single call to `hist()` with an admissible argument, which allows targeted messages for anticipated mistakes (Figure 7). R errors are displayed in the cell as in an ordinary R session (Figure 8).

Q6 — Add a color to the histogram

Re-draw a histogram of `ToothGrowth$len` and set any bar color using the `col=` argument.

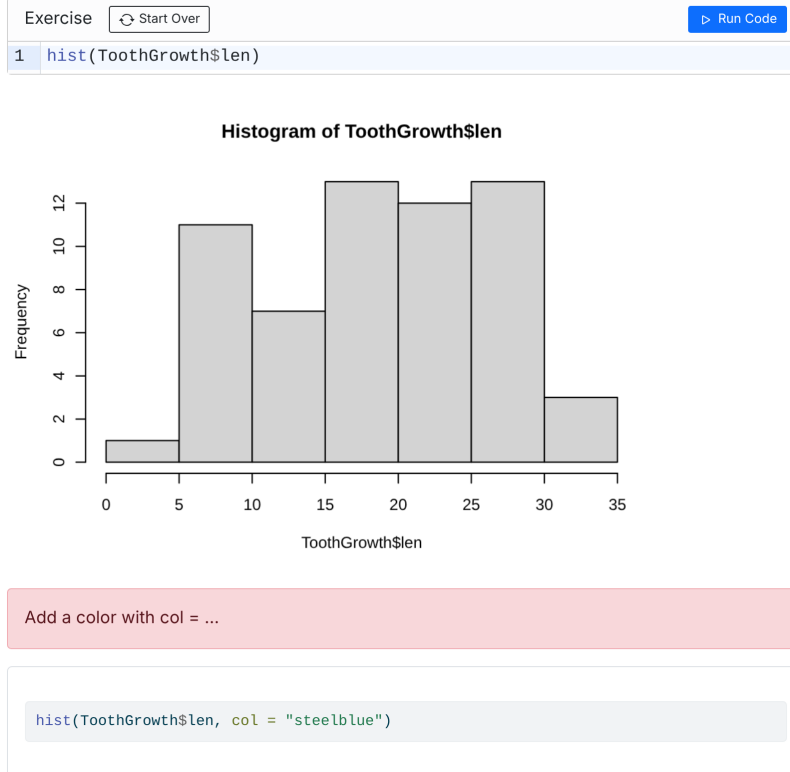


Figure 7: An exercise in which the check detects a missing `col` argument, followed by the hint and the solution.

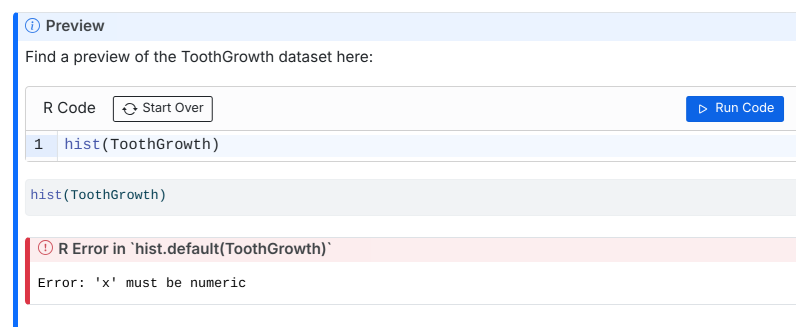


Figure 8: An R error displayed in an interactive cell of the graded tutorial.

Checks may also be too permissive. The third exercise asks for the mean of `c(2, 4, 6, 8, 10)`, but the check only compares the stored value with 6, so the median of a different vector is accepted (Figure 9). Checks that compare values should therefore anticipate incorrect approaches, for instance by also inspecting the code for a call to `mean()`; we return to this point in Section 7.

Q3 — Find the average

Compute the mean of the vector `c(2, 4, 6, 8, 10)`.
Your answer should be stored in a variable called `avg_val`.

Exercise ↺ Start Over 💡 Show Hint ▶ Run Code

1 `avg_val <- median(c(2, 4, 6, 8, 11))`

✔ Correct! You computed the mean successfully.

Figure 9: A false positive: an answer based on the median of a different vector is accepted because only the final value is checked.

4 Strengths and limitations of the webR approach

The main strength of the approach is that students can run R without installing software, which removes a frequent source of difficulty at the start of a course and on restricted machines. Instructors write materials in Quarto or R Markdown, a single source can produce slides, notes and exercises, and hosting is free and requires no server administration. Exercises provide immediate feedback, and since checks are written in R, the feedback can be tailored to anticipated mistakes. The approach is therefore well suited to live demonstrations, in-class activities and simulations, pre-lab preparation and low-stakes self-assessment.

Without additional infrastructure, the approach has the following limitations.

- *Summative assessment.* Solutions are embedded in the page source, and the checks are sent to the browser, where grading takes place. Students who inspect the page can therefore obtain the answers.
- *Recording student work.* Results are displayed to the student but are neither stored nor transmitted to the instructor, and there is no integration with a learning management system.
- *Persistence.* In our tests, edits to exercises were lost when the page was reloaded.
- *Packages and computation.* Only packages with WebAssembly builds are available, and computations run on the student's device, which limits the size of data and the computational load.
- *Interactive features.* On hosts that do not send specific HTTP headers, such as GitHub Pages, functions requiring keyboard input (e.g., `readline()` or the debugger) are unavailable.
- *Layout.* Wide output may be truncated (Figure 5) and tall plots may overflow slides (Figure 3).
- *Maturity.* webR and both extensions are under active development (Stagg et al. 2023b), and the two extensions use different chunk syntax.

5 Interactive tutorials for learning and assessment using learnR

5.1 Main initial takeaways

- Best use is for interactive, low stakes tutorials.

- Supports active learning since students learn by running code.
- Good for formative assessment: useful for practice, readiness checks, lab prep, and review rather than formal high-stakes grading.
- Works best for smaller code tasks to simplify feedback through `gradethis`.
- Immediate feedback is available with `gradethis`.
- Student work can be tracked easily through `learnrhash`, as long as you need an existence proof and not a formal evaluation.
- The choice of hosting matters. Static tutorial `learnr` tutorials materials or pre-rendered R Markdown pages can be shared through GitHub Pages, but standard live interactive `learnr` tutorials require a Shiny-capable environment.

Some issues to think about:

- There are scoping issues which require careful planning when designing tutorials.
- Security is potentially an issue with regards to what you can do, particularly if you are serving the `learnr` tutorial from a server.
- It requires work to convert output from `learnr` hash into readable content for more formal grading.
- For questions where the next part of a question depends on the previous part, you should give the student the answer.
- `learnrhash` relies on Shiny server logic to collect the tutorial state and generate the copy-paste hash (`learnrhash` can not be implemented on GitHub pages as far as I am aware)

6 Autograding of code chunks from Rmarkdown and Quarto documents

7 Designing tutorials and assessment

In the following section, we focus on some elements that are idiosyncratic to **R**, although much of the discussion has similar implications for any other programming language.

- A simple way of getting around the cheating problem is through individualized data generation for which the procedure/script remains the same but answers differ. Of course, the drawback of this choice is that scripts could be shared between students. In data analysis problems, it is also possible to randomize the order of covariates, their name and their selection.
- Asking the right questions: determining the variables and setup (e.g., test statistic, one-sided versus two-sided test, confidence level, conclusion) that can be randomized.
- Generating context using generative AI.
- Validating the results (e.g., numerical tolerance to rounding, differences for p-values for one-sided versus two-sided).

Summative assessment of student answers

- Open-ended questions are best checked through multiple choice questions that force users to generate plots or look at summary statistics and model output.
- Focus on invariants and account for students degrees of freedom: e.g., in regression models, we can't check formulas because variable names may differ following data

wrangling, the syntax and order of variables may be different (in **R** formula notation, `a*b` versus `b + a + a:b`). Likewise, contrast specification may change, as is the default engine class (e.g., `lm` vs `glm` with `family=gaussian`) for fitting these models. We can however compare sample sizes, span of projection (hat) matrices, linear contrasts, tests comparing nested models, residuals and predictions, etc.

- Depending on how prescriptive instructors are (for example, with naming of variables during tidying setup), we can validate more output.
- Conversely, checks that only compare a final value can accept incorrect reasoning when a wrong approach happens to produce the same number, as in the example of Figure 9. Combining a check on the value with a check on the code (e.g., the functions called), or choosing inputs for which common mistakes give distinct answers, reduces such false positives.

Students could simply save certain objects as lists and submit a `.RData` or a CSV file with their short answer. Automated checks can verify class or attributes, and check existence of the variable names; settings may also be set so that the environment is saved at the end of each question. This requires that the answers be numerical, or TRUE/FALSE. If the inputs are randomized, the instructor must run every data version to compare student answers with the solution key.

If students submit a Quarto file that generates an html/PDF report. consider chopping a Quarto file with headers defining questions and sub-questions to facilitate extraction of code or run a text parser to extract comments or text.

8 Evidence from the classroom

 Caution

The list below has not been thoroughly checked and is not exhaustive.

8.1 `learnr`

Table 3: Published evidence on the use of `learnr` in teaching.

| Source | Evidence type | Classroom / teaching | |
|---|------------------------------------|--|--|
| | | context | Summary |
| Stoudt, Scotina, & Luebke,
“Supporting Statistics and
Data Science Education
with <code>learnr</code> ” (Stoudt et al.
2022) | Peer-reviewed
education article | Statistics and data science
education | Discusses classroom uses of <code>learnr</code>
tutorials for statistics and data science
instruction. |

| Classroom / teaching | | | |
|--|------------------------------------|--|--|
| Source | Evidence type | context | Summary |
| Engels, Grosjean, & Artus, “Teaching data science to students in biology using R, RStudio and Learnr” (Engels et al. 2023) | Peer-reviewed classroom study | Biology students learning data science | Analyzes three years of course data and describes learnr tutorials as part of the course design. |
| Aberson, “Building Interactive Tutorials for Teaching Psychological Statistics Online with learnr” (Aberson 2021) | Peer-reviewed teaching article | Psychological statistics | Describes building online interactive tutorials with videos, quizzes, and R analysis space using learnr . |
| Stoudt, “Using learnr Tutorials in an Intro Stats Class as Pre-Labs” (Stoudt 2021) | Instructor classroom report / blog | Introductory statistics pre-labs | Practical classroom account of using learnr tutorials before lab sessions. Not peer-reviewed. |

8.2 webR

Table 4: Published evidence on the use of **webR** in teaching.

| | | Classroom / teaching | |
|--|--|----------------------------|---|
| Source | Evidence type | context | Summary |
| Perkel, “No installation required: how WebAssembly is changing scientific computing” (Perkel 2024) | Nature technology feature | Education-related use case | Discusses WebAssembly scientific computing and includes browser-based R as a way to reduce installation barriers. Not a classroom-outcomes study. |
| Rennie, “Teaching statistics interactively with webR” (Rennie 2023) | Conference talk / teaching demonstration | Statistics teaching | Demonstrates how webR can be used to make teaching materials more interactive with live coding examples. Not peer-reviewed. |

| Classroom / teaching | | | |
|---|---|--|--|
| Source | Evidence type | context | Summary |
| White, “A brief introduction to using WebR and Quarto for client-side interactive lesson material” (White 2023) | Teaching blog / technical demonstration | Client-side interactive lesson materials | Shows how webR and Quarto can be used to let students run R code in lesson materials without installing R. Not peer-reviewed. |
| Ward, “stan-playground: Run Stan models directly in your browser” (Ward et al. 2026) | Peer-reviewed software paper | Browser-based statistical modeling for instruction | Not a classroom study of webR , but relevant to browser-based statistical computing for instruction; discusses integration with webR . |

8.3 Feedback from implementation in our courses

! Important

To be completed by the instructors who used the materials in 2025–2026.

For each course, please report the course and level, approximate enrolment, the materials used and how they were used, student feedback or completion, and the problems encountered. Relevant courses include:

- STA258: Statistics with Applied Probability, at the University of Toronto Mississauga [SUB-SUB-SECTION PROVIDED] ,
- The activities and simulations at St. Francis Xavier University
- Quarto slides at the University of British Columbia.
- Implementation at HEC Montreal?

Testing the template for this paper identified several practical points:

- Students should be advised to wait until webR reports that it is ready, and to expect a delay on the first visit;
- Work in exercises is not saved when the page is reloaded;
- Each page should be inspected in a browser after rendering, since wide output and tall plots may not fit (Figures 3 and 5);
- Checks should be tested against common incorrect answers before release (Figure 9).

8.3.1 Implementation at the University of Toronto Mississauga

STA258: Statistics with Applied Probability is an introductory statistics course at the University of Toronto Mississauga. The materials for STA258 have been developed as

modular components in stages since 2025 (Table 5). An e-book for the course was first created in Summer 2025, without webR. Interactive tutorials using webR and `learnr` were then developed in Fall 2025 (https://nishanmudalige.github.io/STA258_Tutorials/). In Winter 2026, quizzes using webR were created and implemented in the course; these quizzes can be integrated into the Canvas learning management system. The tutorials developed in Fall 2025 were also used in that term: students completed the questions live during tutorial sessions, which were moderated by a teaching assistant (TA). Quarto slides were developed in Summer 2026, while the tutorials remained in use. The complete set of materials is undergoing testing and quality control in Fall 2026, and the redesigned course will go live in Winter 2027.

Table 5: Development and use of webR materials in STA258 at the University of Toronto Mississauga.

| Term | Development | Use in STA258 |
|-------------|--|--|
| Summer 2025 | E-book (without webR) | |
| Fall 2025 | Tutorials using webR and <code>learnr</code> | |
| Winter 2026 | Quizzes using webR, compatible with Canvas | Quizzes implemented; tutorials completed live in TA-moderated sessions |
| Summer 2026 | Quarto slides | Tutorials in continued use |
| Fall 2026 | Testing and quality control | |
| Winter 2027 | | Redesigned course goes live |

Anecdotal feedback from TAs and students has been positive. Both groups value the

live, active-learning component of the tutorial sessions. TAs have observed that students' coding improves over time as they progress through the scaffolded exercises. TAs also appreciate that, since no software needs to be installed, tutorial time is no longer spent initializing R sessions or troubleshooting problems with set-up, installation or reading data files. Students have reported that the exercises are interesting and closely related to the class content. This feedback was gathered informally rather than through a structured evaluation, and should be interpreted accordingly.

8.3.2 Implementation at St. Francis Xavier

To be completed by Kyran

8.3.3 Implementation at the University of British Columbia Okanagan

To be completed by Irene

8.3.4 Implementation at HEC Montréal

To be completed by Léo if webR, learnr, or related tools are implemented.

9 Discussion

With webR and the associated Quarto extensions, instructors can publish slides, activities and auto-graded exercises as static web pages in which students run R without installing software. The course skeleton in Section 3 shows that a complete set of course materials can be built in this way with tools already familiar to many statistics instructors, and that

exercises with staged hints, solutions and tailored feedback can be written in ordinary R code.

The implementation of these tools in STA258 at the University of Toronto Mississauga (Section 8.3) shows how they can be adopted in an existing course. Rather than redesigning the course at once, the materials were developed as modular components over several terms (Table 5): tutorials were introduced first, followed by quizzes and lecture slides. This incremental approach allowed each component to be used and refined in the classroom before the next was added, and may serve as a model for instructors who wish to adopt webR gradually. Integrating the webR quizzes into the Canvas learning management system also kept the activities within an environment already familiar to students.

The anecdotal feedback from STA258 is consistent with the strengths identified in Section 4. Removing installation and set-up from tutorial sessions allowed TAs to devote their time to statistical content and to students' reasoning rather than to technical troubleshooting. The combination of live, TA-moderated sessions with automated checks appears to be effective: the checks provide immediate feedback on routine errors, while the TA can address conceptual difficulties as they arise. TAs also observed that students' coding improved over the term as they progressed through the scaffolded exercises, which suggests that a sequence of short exercises of increasing difficulty is well suited to an introductory course. Students valued exercises that were closely related to the class content, which supports building exercises around the data sets and methods covered in lectures rather than around generic programming tasks. These observations are based on informal feedback from a single course, and a structured evaluation of learning outcomes is required before firmer conclusions can be drawn.

[St. Francis Xavier University, to be completed by K. Cupido: summarize the lessons learned]

from using the webR activities and simulations in class, including what worked well, what did not, and how the experience compares with that in STA258.]

[University of British Columbia Okanagan, to be completed by I. Vrbik: summarize the lessons learned from using Quarto slides with embedded webR cells in lectures, including student reactions and any practical difficulties.]

[HEC Montreal, to be completed by L. Belzile if webR, learnr or related tools are implemented: summarize the context of use and the main lessons learned.]

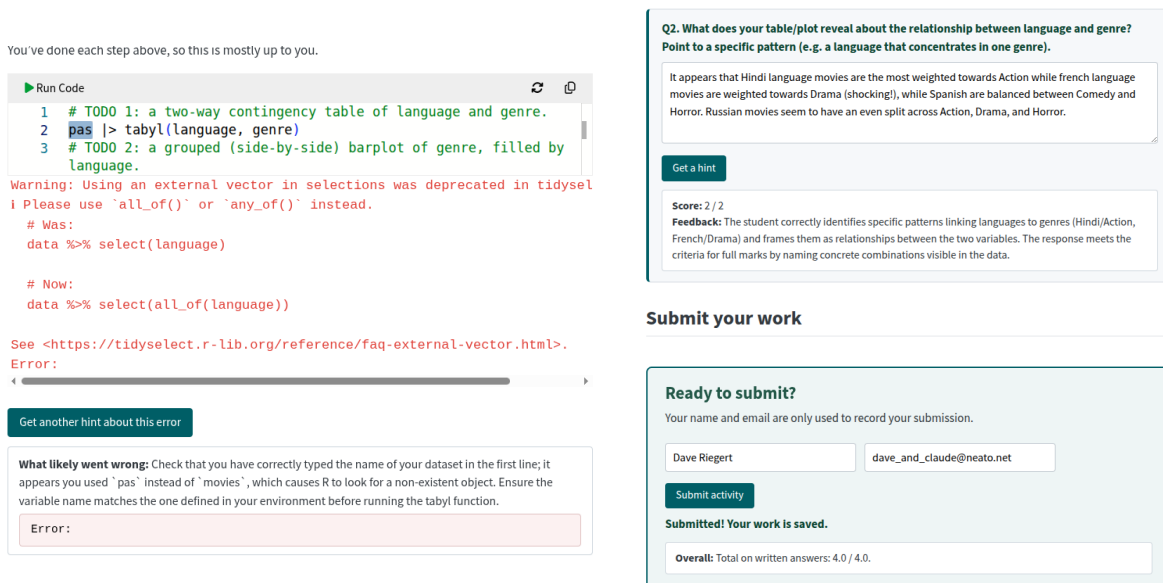
[Other co-authors: add any further experiences, lessons learned or recommendations from implementing these tools in your own courses.]

The approach complements, rather than replaces, the other tools reviewed here. It is best suited to formative, low-stakes use; summative assessment, recording of student work and analyses requiring packages without WebAssembly builds still call for server-based `learnr` tutorials or the autograding workflows of Section 6. In all cases, the quality of automated feedback depends on the design of the checks (Section 7). Future work includes completing the classroom evaluation in Section 8.3, conducting a structured evaluation of the redesigned STA258 once it is offered in Winter 2027, adding a submission mechanism compatible with static hosting, and adding randomized exercises to the template.

10 Possible extension: feedback from large language models

[Extension written based on the content in Dave’s email on LLM’s. Should we include this or not?]

The course skeleton does not use generative AI. Co-author D. Riegert developed a separate prototype of webR exercises in which a large language model (LLM) interprets R errors (Figure 10a), provides hints, and scores written answers against a rubric before submission (Figure 10b). The screenshots in Figure 10 were provided by D. Riegert. The prototype relies on locally hosted models on a private network; wider deployment would require institutional support for hosting the model and for storing student information. Alternatively, WebLLM (Ruan et al. 2024) runs models of up to approximately 14 billion parameters in the browser, so that no student data leave the device, at the cost of requiring sufficient storage and hardware on the student’s machine.



(a) Interpretation of an error

(b) Feedback on a written answer

Figure 10: Screenshots of the LLM prototype by D. Riegert, which is not part of the course skeleton.

Disclosure statement

N. Mudalige, a co-author of this paper, developed the STA258 tutorial website (https://nishanmudalige.github.io/STA258_Tutorials/), from which the graded tutorial in Figures 6 to 9 is taken. The screenshots of the LLM prototype in Figure 10 were provided by co-author D. Riegert. All other screenshots were taken from the CanCOTS course skeleton, whose materials were contributed by the authors.

Supplementary material

The source of the course skeleton is available at https://github.com/nishanmudalige/CanCOTS_2025_Interactive_R_Tutorials, and the rendered site at https://nishanmudalige.github.io/CanCOTS_2025_Interactive_R_Tutorials/course_skeleton/index.html.

References

- Aberson, C. L. (2021), ‘Building interactive tutorials for teaching psychological statistics online with learnr’, *Technology Innovations in Statistics Education* **13**(1).
- Aden-Buie, G., Chen, D., Grolemond, G., Rossell Hayes, A. & Schloerke, B. (2025), *gradethis: Automated Feedback for Student Exercises in ‘learnr’ Tutorials*. R package version 0.2.14.
URL: <https://pkgs.rstudio.com/gradethis/>
- Aden-Buie, G., Schloerke, B., Allaire, J. & Rossell Hayes, A. (2025), *learnr: Interactive Tutorials for R*. R package version 0.11.6.
URL: <https://CRAN.R-project.org/package=learnr>

- Allaire, J. J., Teague, C., Scheidegger, C., Xie, Y., Dervieux, C. & Woodhull, G. (2026), *Quarto*. Version 1.10.
URL: <https://quarto.org>
- Balamuta, J. J. (2023), ‘quarto-webr: Embedded webR code cells for Quarto documents’, <https://quarto-webr.thecoatlessprofessor.com/>. Quarto extension.
- CanCOTS 2025 Interactive Tutorials Working Group (2025), ‘CanCOTS 2025 interactive R tutorials: Course skeleton with slides, activities and tutorials’, https://github.com/nishanmudalige/CanCOTS_2025_Interactive_R_Tutorials. Live site: https://nishanmudalige.github.io/CanCOTS_2025_Interactive_R_Tutorials/course_skeleton/index.html.
- Engels, G., Grosjean, P. & Artus, F. (2023), ‘Teaching data science to students in biology using R, RStudio and learnr: Analysis of three years data’, *Foundations of Data Science* **5**(2), 266–285.
- Galassi, A. & Vittorini, P. (2021), Automated feedback to students in data science assignments: Improved implementation and results, *in* ‘CHIItaly 2021: 14th Biannual Conference of the Italian SIGCHI Chapter’, ACM, Bolzano, Italy, pp. 1–8.
- Horst, A. M., Hill, A. P. & Gorman, K. B. (2020), *palmerpenguins: Palmer Archipelago (Antarctica) Penguin Data*. R package version 0.1.0.
URL: <https://allisonhorst.github.io/palmerpenguins/>
- Jahodová Berková, A. & Němec, R. (2020), ‘Teaching theory of probability and statistics during the Covid-19 emergency’, *Symmetry* **12**(9), 1577.
- Perkel, J. M. (2024), ‘No installation required: How WebAssembly is changing scientific computing’, *Nature* **627**(8003), 455–456.

Pierson, S. (2023), ‘Data analytics, data science degrees see large increases in 2022’, Amstat News.

URL: <https://magazine.amstat.org/blog/2023/12/01/degreesstats2022/>

Poulos, A. & Mahony, M. J. (2008), ‘Effectiveness of feedback: The students’ perspective’, *Assessment & Evaluation in Higher Education* **33**(2), 143–154.

Pyles, C. (2025), ‘Otter-grader: A lightweight autograder for Python and R assignments’, <https://otter-grader.readthedocs.io/>. UC Berkeley Data Science Education Program. R support via the ottr package.

R Core Team (2025), *R: A Language and Environment for Statistical Computing*, R Foundation for Statistical Computing, Vienna, Austria.

URL: <https://www.R-project.org/>

Rennie, N. (2023), ‘Teaching statistics interactively with webR’, Lightning talk.

URL: <https://nrennie.rbind.io/talks/rss-rose-statistics-education-2023/>

Ricci, F. Z., Medina, C. M. & Dogucu, M. (2024), ‘Automated grading workflows for providing personalized feedback to open-ended data science assignments’, *Technology Innovations in Statistics Education* **15**(1).

Ruan, C. F. et al. (2024), ‘WebLLM: A high-performance in-browser LLM inference engine’, *arXiv preprint arXiv:2412.15803*.

Rundel, C. & Çetinkaya-Rundel, M. (2026), *ghclass: Tools for Managing Classes on GitHub*. R package version 0.4.2.

URL: <https://CRAN.R-project.org/package=ghclass>

- Stagg, G. W. (2024), ‘Quarto live: Interactive code blocks and exercises in Quarto using WebAssembly’, <https://r-wasm.github.io/quarto-live/>. Quarto extension.
- Stagg, G. W., Henry, L. et al. (2023a), ‘Installing R packages in webR’, <https://docs.r-wasm.org/webr/latest/packages.html>. Public repository of WebAssembly R packages at <https://repo.r-wasm.org/>.
- Stagg, G. W., Henry, L. et al. (2023b), ‘webR: The statistical language R compiled to WebAssembly via Emscripten’, <https://docs.r-wasm.org/webr/latest/>. Documentation and source code at <https://github.com/r-wasm/webr>.
- Stoudt, S. (2021), ‘Using learnr tutorials in an intro stats class as pre-labs’, Blog post.
URL: <https://sastoudt.github.io/posts/2021-06-05-learnr-tutorials-intro-stat/>
- Stoudt, S., Scotina, A. D. & Luebke, K. (2022), ‘Supporting statistics and data science education with learnr’, *Technology Innovations in Statistics Education* **14**(1).
- Timbers, T., Campbell, T. & Lee, M. (2022), *Data Science: A First Introduction*, Chapman and Hall/CRC, Boca Raton, FL.
URL: <https://datasciencebook.ca/>
- Ward, B., Soules, J. & Magland, J. (2026), ‘stan-playground: Run Stan models directly in your browser’, *Journal of Open Source Software* **11**(118), 9531.
- White, E. (2023), ‘A brief introduction to using WebR and Quarto for client-side interactive lesson material’, Jabberwocky Ecology (blog).
URL: <https://jabberwocky.weecology.org/2023/03/13/a-brief-introduction-to-using-webr-and-quarto-for-client-side-interactive-lesson-material/>

Woodard, V., Lee, H. & Woodard, R. (2020), ‘Writing assignments to assess statistical thinking’, *Journal of Statistics Education* **28**(1), 32–44.

Zeileis, A., Umlauf, N. & Leisch, F. (2014), ‘Flexible generation of e-learning exams in R: Moodle quizzes, OLAT assessments, and beyond’, *Journal of Statistical Software* **58**(1), 1–36.

Zimmerman (2020), ‘[reference to be completed: Zimmerman (2020), webwork]’.